MAPLESURE INSURANCE · AMS CONSOLE

S3 Enhancement Delivery

Test & walkthrough guide — all scenarios

Generated 3 August 2026 · repo `ams-s3-demo` · branch `s3-console-stage-routes`
Companion docs: `docs/design/S3_DESIGN.md` (technical design) · `demo/DEMO_TEST_GUIDE.md` (presenter script) · `repos/README.md` (the target repositories, and onboarding a new one)

Environment

Six processes make up the full environment. The console and the app a given CR targets must be running; the rest can stay down if you are not showing that CR.

PORT	SERVICE	URL	START COMMAND (TERMINAL BY TERMINAL)
8000	FastAPI backend the engine — never on screen	`localhost:8000`	`apps/run-console.sh` `starts 8000 and 5173 together`
5173	**AMS Console** — start here login, then `/s3`; managers also get `/admin`	`localhost:5173`	`apps/run-console.sh`
8501	PolicyCore portal (CR-041 / CR-042 target) "the client's app"	`localhost:8501/sl_policycore` `base path — the bare root 404s`	`apps/run-policycore.sh`
8081	ClaimsPortal Contracts Team console (CR-043)	`localhost:8081`	`apps/run-policy-service.sh`
8082	ClaimsPortal Claims Team console (CR-043)	`localhost:8082`	`apps/run-claims-service.sh`
8083	EnrolDirect (CR-045 target) online enrolment channel	`localhost:8083`	`apps/run-enroldirect.sh`

Both ClaimsPortal services together, in one foreground terminal: `demo/run_s3_claimsportal.sh`. It traps `kill 0` on exit, so use the two separate commands above if you want them in independent background terminals. `demo/run_mockapp.sh` is an equivalent launcher for the portal — same base path, same `PYTHONPATH` export — kept because teammates and the admin panel both invoke it by name.

> **YOU NO LONGER NEED A TERMINAL FOR MOST OF THIS**
>
> Log in as **Manager** and open `/admin`. It shows which of the four target apps are answering on their port, starts or stops them, clears logs, runs the reset scopes, and onboards a repo by writing its `.s3targets.json`. Service status is a plain TCP connect — no `ps`, no `lsof` — so it works on a locked-down host.
>
> Two limits worth knowing before you rely on it mid-run. There is **no service id for the console**: it cannot restart the process serving your request, and naming it is a 400 rather than an attempt. And a manifest the panel just wrote **needs a console restart** before that target registers.

Where the code lives

Two folders, and the split is load-bearing. `repos/` holds the three target repositories — `policycore`, `claimsportal`, `enroldirect` — which are what S3 *changes*. `apps/` holds the console and the launch scripts, which are what does the changing. Dropping a directory into `repos/` with a `.s3targets.json` manifest registers it as a target at next start, with no edit to any registry file; putting its CR under `crs/` opens the board ticket. See `repos/README.md` for the manifest contract and the four onboarding steps — it is short, and it is the authority.

Direct URLs for showing raw data

URL	SHOWS
`localhost:8000/docs`	OpenAPI browser for the whole S3 API
`localhost:8081/api/policies`	All ClaimsPortal policies as JSON
`localhost:8081/api/policies/MS-1004`	One policy — the clearest place to show `deductible` appearing
`localhost:8082/api/claims`	Submitted claims with decision and `payableAmount`
`localhost:8083/api/analysis/prospect-impact`	EnrolDirect's both-policies comparison — the analysis CR-045 acts on

> **THREE THINGS THAT WILL BITE YOU**
>
> **Never run uvicorn with `--reload`.** The pipeline writes `.py` files into the tree uvicorn watches; the reload wipes the in-memory session store and every request then 401s while the UI still looks logged in.
>
> **The ClaimsPortal processes serve whatever was last running.** There's no build step (plain Python), but `uvicorn` is deliberately started without `--reload`, so an Apply changing the source does not take effect until you restart both services. A process already running survives the reset entirely: it keeps serving the *post-CR* app while the source sits at baseline, and both ports answer 200 the whole time. Port-is-up is not process-is-current — check the process start time against your reset time.
>
> **Start the portal via a launch script, never `streamlit run` by hand.** `apps/run-policycore.sh` and `demo/run_mockapp.sh` both export `PYTHONPATH="$PWD"` first and pass the same base path; either is fine, neither is optional. Streamlit otherwise puts `repos/policycore/` on `sys.path` instead of the repo root, and the app dies at `ModuleNotFoundError: No module named 'common'`. This one hides: a portal already running from a bad launch keeps serving its last good render until something re-executes the script — and a reset rewriting `app.py` is exactly that, so the error surfaces *during the reset* and looks like the reset broke it. It didn't.

Logins

Fictional roster; passcode is `1001 + roster position`.

NAME	PASSCODE	ROLE IN THESE TESTS
Ravi Kumar	1001	Developer — owns AMS-102, AMS-103
Elena Cruz	1002	Second engineer — owns AMS-098
Priya Nair	1003	Tester — owns AMS-101; QA hand-off target
Tom Becker	1004	Second tester

Manager	`9000`	Manager view

Seeded board

Verified against a live `GET /api/s3/jira/board` on 3 August 2026.

TICKET	CR	STATUS	ASSIGNEE	TARGET	ORIGIN
`AMS-101`	`CR-2026-041`	QA	Priya Nair	policycore — plan tier upgrade	seeded
`AMS-102`	`CR-2026-042`	In Progress	Ravi Kumar	policycore — amendment priority	seeded
`AMS-103`	`CR-2026-043`	To Do	Ravi Kumar	claimsportal — claims deductible	seeded
`AMS-104`	`CR-2026-044`	To Do	Ravi Kumar	policycore — flag urgent amendments	seeded
`AMS-098`	—	Done	Elena Cruz	none (ad-hoc path)	seeded
`AMS-1045`	`CR-2026-045`	To Do	*unassigned*	enroldirect — prospect access	**picked up from `crs/`**

> **AMS-1045 IS NOT SEEDED — IT IS DERIVED, AND THAT IS A BEAT**
>
> A `.md` file dropped into `crs/` opens a board ticket on its own. The key is a pure function of the CR identifier (`CR-2026-045` → **AMS-1045**), so it survives restarts and resets with nothing to persist, and it sits in a band the hand-seeded tickets cannot reach. It lands **unassigned** on purpose — that is what puts it in front of the manager, who sees exactly the unassigned ones with an Assign control.
>
> Assignment used to be set-once, enforced only by hiding the button. A manager can now reassign or unassign, and `POST /s3/jira/assign-ticket` is **manager-only server-side** (`require_manager`) — it previously had no role check at all. If you rehearse the assign flow, the board you hand to the next presenter will show whoever you picked, not *unassigned*; reset the `tickets` scope to put it back.

Reset between runs

The pipeline mutates real files. Stop the services first, then run these in **this order** — the CR-042 reset must come *after* `reset_s3.sh`, because it restores the superset PolicyCore state that leaves both AMS-101 and AMS-102 ready to run.

```
demo/reset_s3.sh              # CR-041 + shared state: .cache/llm, out/, events, Jira caches
demo/reset_s3_endorsement.sh  # CR-042 baseline  (MUST run after reset_s3.sh)
demo/reset_s3_claimsportal.sh # CR-043 target, from repos/claimsportal/.baseline/
demo/reset_s3_enroldirect.sh  # CR-045 target, from repos/enroldirect/.baseline/
```

`reset_s3_endorsement.sh` keeps its pre-reskin filename deliberately — teammates invoke it by name, and only its contents changed. The same reset scopes are on the manager's `/admin` panel if you would rather not leave the browser; there is no "reset everything" scope, by design — you name one.

Then restart the services. Hard-refresh the browser tab — the console caches per-ticket state in `localStorage` and drops it when the reset marker changes.

THE TWO POLICYCORE RESETS RESTORE FROM HEAD

`demo/reset_s3.sh` and `demo/reset_s3_endorsement.sh` restore source with `git checkout HEAD -- repos/…`, so **HEAD must already carry the paths they name**. Move a target directory and both stop at the checkout with *"pathspec did not match"* until the move is committed — committing it is the whole fix, nothing in the scripts needs changing.

The admin panel checks that before it runs anything and reports a named `reset_blocked_reason` rather than surfacing a raw git error out of a button. The ClaimsPortal and EnrolDirect resets copy from their committed `.baseline/` snapshots, so they never depend on HEAD.

EXPECTED AFTER A RESET

`git status` shows `repos/policycore/{app,core/db,core/amendments,core/models}.py` as modified. That is **correct** — repo HEAD is committed in a post-CR-2026-042 state, so the pre-CR baseline necessarily differs from it.

Resets also clear `.cache/llm`, so impact analysis, design doc and release notes make live provider calls again (~10–20 s each on first click). Codegen and testgen always replay — instant and free. Run `demo/warm_s3_cache.sh` if you want the narrative beats instant too.

Before → after: what changes on screen

The "show the client their own app changing" moment for each scenario. Have this open while presenting.

SCENARIO	WHERE TO LOOK	BEFORE — BASELINE	AFTER APPLY	TO SEE IT
AMS-101 CR-2026-041	Portal :8501/sl_policycore Policy Detail	No plan tier anywhere in the app.	Tier per contract (Standard / Premium / *the name the audience picked*), upgrade control on the detail view, monthly contribution recalculates and persists.	Browser refresh
AMS-102 CR-2026-042	Portal :8501/sl_policycore Request a Contract Amendment	Exactly 5 fields — amendment type, describe the requested change, effective date, contact phone, contact email.	A 6th field, **Priority** (Standard / Urgent, defaults Standard). Existing submit flow unchanged.	Browser refresh
AMS-103 CR-2026-043	Claims :8082 and Policy :8081	Policies carry no deductible. An **$80 claim on MS-1004 → ACCEPTED**.	Policies gain a deductible (MS-1001 500, MS-1002 1000, MS-1003 500, MS-1004 100). The same **$80 claim on MS-1004 → REJECTED_BELOW_DEDUCTIBLE**. A **$1,200 claim on MS-1001 → ACCEPTED, `payableAmount` 700**.	**Restart both services**

AMS-1045 CR-2026-045	EnrolDirect :8083 Access check	A prospect is refused, and the refusal is an omission rather than a rule: `"category PROSPECT has no online enrolment preference"`, `requiredPreference: null`. **0 of 6 prospects admitted.**	A prospect resolves through the configured policy to the **Guest** preference. AP-4003 and AP-4008 are admitted with `authorisingPreference: "Online Enrolment – Guest"`; the other four are still refused, each for its own named reason (member-only sponsor, no preferences, lapsed contract). **2 of 6 admitted.**	**Restart the service**

> **WHY CR-045 IS THE DIFFERENT ONE**
>
> The other three CRs add something. **CR-2026-045's baseline is a removal** — the checked-in source is the state after the impact analysis and before anyone acted on it, so the "before" is a refusal nobody chose. The CR settles a classification rather than building a feature, which is why the honest before/after is a decision changing, not a control appearing.
>
> Second thing worth saying out loud: `repos/enroldirect/impact.py` is in the target's **core files but deliberately not in its codegen allowlist**. The model must read the analysis to understand the change and must not edit it — the analysis has to keep sizing *both* options after one is adopted. Core-file recall gets it into the prompt; the allowlist keeps it out of the diff, and this target has its own validator that fails loudly if a read-only file comes back modified.

> **THE EASIEST WAY TO MAKE A WORKING RUN LOOK BROKEN**
>
> The two PolicyCore scenarios need only a browser refresh — Streamlit re-executes `app.py` on each run. **Scenario 3 does not.** The running processes serve whatever was loaded at startup (no `--reload`), so a just-applied CR is invisible on :8081/:8082 until you restart. Apply the change, switch to :8082, and nothing has changed. Ctrl-C both terminals, then `demo/run_s3_claimsportal.sh`.

Confirm the baseline is genuinely "before"

Run from the repo root after the resets, before you present. Every line must match the comment. Re-derived and re-run against the GRS-reskinned tree on 3 August 2026 — the old `coverage_tier` / `COVERAGE_TIERS` grep now passes vacuously, because those symbols no longer exist under any state of the repo. The CR-041 sentinel is `PLAN_TIERS` / `upgrade_tier`, not `plan_tier`: the field exists on `Policy` before the CR and the portal's contract header displays it, so `plan_tier` alone is a false positive. What CR-041 adds is the tier *ladder* and the upgrade call.

```
grep -c 'PLAN_TIERS\|upgrade_tier' repos/policycore/app.py  # 0       — no CR-041 yet
grep -c 'selectbox("Priority"' repos/policycore/app.py      # 0       — no CR-042 yet
ls repos/policycore/core/tiers.py                           # absent  — CR-041 creates it
ls repos/claimsportal/claims_service/claim_rules.py         # absent  — CR-043 creates it
ls repos/policycore/core/amendments.py                      # PRESENT — CR-042's "before" needs it
grep -c 'DEFAULT_PROSPECT_POLICY' repos/enroldirect/applicants.py   # 0 — no CR-045 yet
curl -s localhost:8081/api/policies/MS-1004                 # no "deductible" key
curl -s -X POST localhost:8083/api/eligibility/check \
  -H 'Content-Type: application/json' -d '{"applicantId":"AP-4003"}'  # "granted": false
ls s3_enhancement/out/ | wc -l                              # 0
```

Two of those are worth reading rather than glancing at. The CR-041 grep is scoped to **`app.py` only**, and its sentinel is `PLAN_TIERS` / `upgrade_tier` rather than `plan_tier`, for the same reason: `core/models.py`

and `core/db.py` carry a `plan_tier` column at baseline by design, so a `plan_tier` grep is not zero and is not a failure. The two ladder symbols appear in no source file at baseline — only in `AGENTS.md` / `CLAUDE.md`, which the greps do not read. And the `AP-4003` check must come back `"granted": false` with `"requiredPreference": null`; a prospect refused for any *other* reason means the EnrolDirect reset did not take.

Then confirm the services are up *and fresh* — every start time must be later than your last reset:

```
for p in 8000 5173 8501 8081 8082 8083; do
  url="http://localhost:$p/"
  [ "$p" = 8501 ] && url="http://localhost:8501/sl_policycore/"   # Streamlit base path
  pid=$(lsof -ti tcp:$p -sTCP:LISTEN | head -1)
  code=$(curl -s -o /dev/null -w '%{http_code}' --max-time 5 "$url")
  printf "%-5s http=%-3s pid=%-6s %s\n" "$p" "$code" "${pid:-DOWN}" \
    "$(ps -o lstart= -p ${pid:-0} 2>/dev/null)"
done
```

Use `-sTCP:LISTEN`. Without it a Chrome tab connected to :5173 matches before the real listener and you read the wrong process's start time. The admin panel's service list answers the up/down half of this without a terminal, but it is a TCP probe only — it cannot tell you a process is *stale*, which is the failure this loop exists to catch.

Scenarios

> **THE SCREEN CHANGED — ARTIFACTS NOW OPEN IN A POP-UP**
>
> Acting on the client's "there is a lot of data on the screen… open it in a pop-up" feedback, every produced **artifact** now opens in a modal rather than rendering inline: release notes, deployment plan, design doc, test scenario table, per-case checklists, mutation diff, traceability matrix. What stays in the main flow is what you steer with — action buttons, verdict lines, token-cost lines. The left-hand stage rail is unchanged.
>
> Practical effect on these scripts: where a step below says "read the X", expect to click through and then close the modal before the next button is reachable. Nothing moved stage to stage.

01 AMS-103 — ClaimsPortal, full pipeline

Start as: Ravi Kumar (1001) → hand off to Priya Nair (1003) · **Time:** ~15 min · **Best single end-to-end run**

1. Open :5173 → *Open application* → click **AMS-103**.
2. **Run AI impact analysis.** It asks up to two clarifying questions — a gap question, then an assumption question. Answer each in the same box.
3. Close the modal → **Stage 1 · Generate the change**.
4. **Apply to repo.**
5. **Stage 2 · Draft design doc** → **Assign tester & move to QA** (Priya Nair).
6. Log out → log in as **Priya Nair** → open AMS-103 → **Stage 3 · Generate tests** → **Run tests** (real pytest, the same runner every target uses).
7. **Prove the tests catch bugs** → **Stage 4 · Draft release notes** → mark Done.

- ☐ After the second answer it *must* produce a final analysis — the two-turn cap is hard.
- ☐ Activity tab logs: clarification requested ×2 → impact analysis drafted → status In Progress.
- ☐ Selection panel reads **Files in this app 5 · used 5** and *"No subsystem doc matched closely enough"* — no `repos/policycore/systems/…` entry.
- ☐ Token panel says *"no saving over whole-app context here"* — the honest branch.
- ☐ Six Python files, each with a one-line reason.
- ☐ Post-apply link reads **"open the Claims Team console"** → :8082 (not the PolicyCore portal).
- ☐ On disk: `claim_rules.py` created, `deductible` present in `policy.py`.
- ☐ Stage 3 was *locked* for Ravi — the developer cannot verify their own change.
- ☐ Per-case checklist parsed from pytest's JUnit XML output, all green.
- ☐ Mutation flips `<=` to `<`; the "claim at exactly the deductible" test goes red; file auto-restored (`git diff` on `claim_rules.py` is clean afterwards).

> **:8081 WILL NOT CHANGE — BY DESIGN**
>
> CR-2026-043 line 50 states *"Both team consoles must keep rendering without changes to their HTML"*, and no `.html` is in the target's codegen allowlist. The policy console's table has no Deductible column, so it looks identical after the CR. The change *is* there — `GET :8081/api/policies` now returns `"deductible":500`.
>
> **The visible proof is on :8082.** Restart the services, then submit a claim against MS-1001 (limit 25 000, deductible 500):

CLAIM AMOUNT	BEFORE THE CR	AFTER THE CR
400	ACCEPTED	REJECTED_BELOW_DEDUCTIBLE
500 — exactly the deductible	ACCEPTED	REJECTED_BELOW_DEDUCTIBLE
1 200	ACCEPTED	ACCEPTED — payable 700 now recorded
99 000	REJECTED_OVER_LIMIT	REJECTED_OVER_LIMIT

02 AMS-101 — the scoring showcase

> **As:** Priya Nair (1003) — the ticket is seeded to her, already in QA, so she runs the whole flow · **Time:** ~10 min

1. Click **AMS-101** → **Run AI impact analysis**.
2. **Stage 1 · Generate** — type an audience-chosen tier name (≤ 24 chars, no quotes or slashes, e.g. *Platinum*).
3. **Apply** → post-apply runs `python -m repos.policycore.core.seed`.
4. **Design doc** → **Generate tests** → **Run** (pytest) → **Mutation check** → **Release notes**.

- ☐ Panel reads **Files in this app 58 · used 4**.
- ☐ Expand "7 other subsystems screened out" → bar chart, struck through: settlement 0.35, enrolment 0.22, risk 0.21, billing 0.20, underwriting 0.19, audit 0.15, reporting 0.12 — all under the 0.45 floor. **52 files never opened** — the 50 Java decoys plus the two Python modules of PolicyCore's own *enrolment* subsystem, which is the better line: screening is a scope decision, not a file-extension filter.
- ☐ Token panel shows a real multiplier against whole-app context (3.4× on this CR — 18,310 estimated file tokens down to 5,336).
- ☐ Your chosen tier name appears verbatim in the generated `tiers.py` — one recording serves any name.
- ☐ Refresh **:8501/sl_policycore** → the portal now has the plan-tier upgrade control, and the monthly contribution moves with the tier.
- ☐ Mutation weakens the same-tier guard `<=` → `<`; the suite goes red, then reverts.

03 AMS-102 — amendment priority

> **As:** Ravi Kumar (1001) → hand off to a tester · Same step sequence as Scenario 01

> **THE DOCSTRING PROBLEM IS FIXED — THIS IS NOW SAFE TO APPLY ON STAGE**
>
> The old warning here said the CR-042 recording stripped roughly six function docstrings from `repos/policycore/core/db.py`, removing 81 lines to add 12. The 3 August 2026 re-record fixed it. Re-verified by diffing the committed recording against the baseline tree: **docstring counts are identical file for file** (`db.py` 40 before and after), and the whole four-file diff is **+16 / −32** rather than +12 / −81.
>
> What is left is cosmetic and worth naming before a reviewer spots it: the model collapses PEP 8's two blank lines between top-level definitions down to one, and it de-indents the continuation line of two docstrings (`core/amendments.py`, `app.py`). Annoying in a diff; not a behaviour change. Model docstring mangling recurs on every re-record — always diff the recording against the baseline after one.

- ☐ All four files show pending changes. (Earlier recordings dropped one as byte-identical; this one does not.)
- ☐ `models.py` gains `priority: str = "Standard"`, `db.py` gains the column and the `INSERT` parameter, `amendments.py` takes it as a keyword argument.
- ☐ If applied: **:8501/sl_policycore** → *Request a Contract Amendment* gains a *Priority* dropdown (Standard / Urgent, Standard default).

04 The review loop

> **Where:** any ticket, Stage 1, after Generate and before Apply

ACTION	EXPECTED
Show diff on a file	Per-file unified diff, collapsible
Ask about this file → *"why did you delete those blank lines?"*	Answers **from its own diff** — admits the deletion instead of denying it. (On CR-042 that is a real, visible change to point at.)
Ask → *"rename that variable to X"*	Returns a revised file; still staged, still not applied
Ask a pure question	Answers in prose and changes **no** code
Add to proposal → `repos/policycore/core/claims.py` + an instruction	File joins the same reviewed diff
Apply a single file	Others stay pending; re-applying is a no-op

- ☐ Throughout: `git status` stays clean until Apply. Nothing reaches the repo before then.

05 Cross-team impact & the dependency gate

> **Needs two logins** · Ravi Kumar (1001) + whoever the new ticket is assigned to

1. As Ravi, open AMS-102 or AMS-103 → **Check for other teams affected**.
2. Confirm a suggested team → a linked Jira ticket is created and assigned.
3. Log in as that assignee → open their ticket → **Mark as Done**.
4. Log back in as Ravi.

- ☐ After step 2, Stage 1 is blocked: *"Waiting on 1 other team to finish their ticket."*
- ☐ After step 3, the gate clears for Ravi — state comes from the append-only events log, not the browser, which is why it crosses separate logins.
- ☐ No ticket is ever created without a human confirming it.

06 Ad-hoc ticket & the clarity gate

> **As:** Elena Cruz (1002) · **Ticket:** AMS-098 — no linked CR, so it routes to the ad-hoc path

- ☐ Runs with **no codebase context** — correctly shows no file-selection panel.
- ☐ Three gates share one two-question budget: overall vagueness → a specific missing detail → repo identity.
- ☐ A repo match below *high* confidence asks *"is this the right repo?"* rather than scoping against a guess.
- ☐ If GitLab is unreachable the repo check is skipped silently and analysis proceeds.

07 Quick-impact chat

> **Where:** "Quick question" box at the top of the S3 page · any login

Try: *"how long would it take to add a beneficiary-change amendment?"*

- ☐ Asks at most **two** clarifying questions, then must deliver a final answer.
- ☐ Final answer carries impact analysis + hour class + P-level + whether a code change is warranted.
- ☐ **Clear** resets the server-side transcript.

08 Problem-record intake — the second CR flavour

> **Run:** `demo/seed_problem_record_ticket.sh` (API must be running)

- ☐ A new ticket appears on the board tagged **origin: problem record** with its `PRB…` id.
- ☐ It runs the identical downstream flow — only the origin tag differs, which is the point: repeated incidents → problem record → CR converges on the same pipeline.

09 GitLab repo scoping — API only, no UI

```
C=$(mktemp); curl -s -c $C -X POST localhost:8000/api/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"name":"Ravi Kumar","passcode":"1001"}' >/dev/null

curl -s -b $C -X POST localhost:8000/api/s3/gitlab/scope-auto \
  -H 'Content-Type: application/json' \
  -d '{"tier_name":"Elite"}' | python -m json.tool
```

- ☐ The AI picks a repo from names and descriptions alone, returning confidence, reasoning and alternates.
- ☐ Below *high* confidence it returns `needs_clarification` instead of guessing.
- ☐ `repo_size` vs `files_reached_llm` shows the funnel economics — only a shortlist is ever fetched, so cost stays flat regardless of repo size.
- ☐ Nothing is ever written back to GitLab.

10 Resilience & regression

```
.venv/bin/python -m pytest tests/ -q                    # 679 collected
.venv/bin/ruff check .                                  # clean
.venv/bin/python tools/verify_s3_live.py --skip-live    # offline architecture gate
```

- ☐ Replay works with networking stubbed out entirely.
- ☐ A mid-stream provider failure falls back to replay invisibly.
- ☐ One recording replays correctly for two different tier names.
- ☐ Reset restores baseline in under 10 s.
- ☐ Core recall never drops a required file; screened-out subsystems are never selected.
- ☐ The four checked-in regression suites pass on their own: `policycore 20 · policycore_enrolment 19 · claimsportal 5 · enroldirect 24`. None of those paths appears in any target's `testgen_allowlist` or `codegen_allowlist` — `tests/test_s3_testrun.py` asserts it, and that assertion is the whole value of the beat.

> **SESSION-DAY RULE**
>
> `tools/verify_s3_live.py --gate 10` runs ten real codegen + testgen + pytest cycles from baseline. A cycle passes only if the **live** path succeeded — no silent replay fallback — and the generated tests ran green. **Require ≥ 9/10, otherwise present in `LLM_MODE=replay` without hesitation.**

11 Onboarding a repo, and the admin panel

As: Manager (9000) · **Where:** `/admin` · **Time:** ~5 min · Answers "what does it take to point this at *our* repo?"

1. Log in as **Manager** → `/admin`. An engineer or tester never sees it — every route depends on `require_manager`, server-side.
2. **Target apps** — the four services with their live port state; start or stop one without a terminal.
3. **Onboard a repo** — writes a `.s3targets.json` manifest for a directory under `repos/`.
4. **Reset environment state** — pick a scope, read its preview, then run it.

- ☐ A source-restoring scope **previews first**: exactly what it restores, what it deletes, and which of those paths are currently dirty. It **409s** rather than overwriting your uncommitted work.
- ☐ There is **no "reset everything"** scope. Seven explicit ones — `policycore`, `claimsportal`, `enroldirect`, `tickets`, `logs`, `proposals`, `caches` — and you name one.
- ☐ Naming the console as a service is a **400, not an attempt** — it cannot restart the process serving the request.
- ☐ A manifest written here **needs a console restart** before that target registers. Say so rather than clicking twice.
- ☐ Drop a `CR-YYYY-NNN.md` into `crs/` and reload the board: the ticket is there, unassigned, keyed off the CR id. That is the whole onboarding story — the four steps are in `repos/README.md`.

Known issues

#	ISSUE	IMPACT	STATUS
1	`demo/reset_s3.sh` and `demo/reset_s3_endorsement.sh` restore with `git checkout HEAD -- repos/…`, and the `apps/` → `repos/` move was uncommitted, so HEAD did not have those paths	Both PolicyCore resets failed with *"pathspec did not match"*	**Fixed** — 3 Aug 2026, by committing the move (`e5af8ed`); all four resets verified to exit 0. The standing rule: a target move must be committed before these two can restore. The admin panel's `reset_blocked_reason` check stays, for the next move.
2	CR-2026-042 codegen recording stripped ~6 function docstrings from `repos/policycore/core/db.py`	AMS-102 diff was dominated by unrelated deletions	**Fixed** — 3 Aug 2026 re-record; docstring counts now identical file for file. Cosmetic blank-line collapsing remains.
3	Subsystem screening was hardcoded to PolicyCore, so the ClaimsPortal target was scored against PolicyCore's decoy subsystems	Panel wrongly reported `legacy_platform/settlement` 0.49 for AMS-103	**Fixed** — target-scoped, 2 regression tests
4	Post-apply link always pointed at the PolicyCore portal	AMS-103 sent reviewers to the wrong app	**Fixed** — per-target app map
5	`POST /s3/jira/assign-ticket` had no role check at all — assignment was gated only by hiding the button	Any logged-in engineer could assign, and assignment was set-once	**Fixed** — `require_manager` server-side; managers can now reassign and unassign
6	AMS-103's Jira card shows `APPLICATION: PolicyCore` though it is a ClaimsPortal CR	Cosmetic inconsistency in seeded data	Open — low priority
7	Gap gate occasionally asks about a detail the CR already states (e.g. the deductible's data type)	One wasted clarification turn	Open — prompt tuning
8	After QA hand-off the console silently re-targets another ticket, since the handed-off one leaves the developer's filtered board	Disorienting mid-run	Open — low priority
9	ClaimsPortal consoles show no deductible column; `payableAmount` is not rendered either	No visual before/after on :8081	By design — CR forbids HTML changes

Troubleshooting

SYMPTOM	CAUSE AND FIX
UI looks logged in but every action 401s	Backend restarted (or was run with `--reload`) and cleared the in-memory session. Log out and back in; restart uvicorn without `--reload`.

:8081 or :8082 refuses to connect	Neither service is running — start with `apps/run-policy-service.sh` / `apps/run-claims-service.sh` (or `demo/run_s3_claimsportal.sh` for both).
Applied a CR but the ClaimsPortal console is unchanged	Stale process — uvicorn runs without `--reload` here, so it's still serving the pre-apply code. Ctrl-C and relaunch both services.
A stage will not open	Not a bug — the grey hint under it names the missing precondition (analysis not run, change not applied, ticket not in QA, or you are not the assigned tester).
Console shows results for a ticket you just reset	`localStorage` cache. Hard-refresh the tab; the reset marker makes the console drop it.
Mutation check returns 409	Generated code drifted from the recording, so no seeded snippet matched. Re-run generate/apply, or re-record. It fails loudly rather than mutating blind.
Analysis is slow on first click	Expected — `.cache/llm` was cleared by the reset. Run `demo/warm_s3_cache.sh` to pre-warm.
A PolicyCore reset dies with `pathspec 'repos/policycore/app.py' did not match`	A target directory was moved and the move is not committed, so those paths are not in HEAD for the script to restore from. Commit the move. Nothing else fixes it.
`codegen returned unexpected file set`	A repository under `repos/` was renamed or moved. The committed recordings contain the exact paths, both as keys and inside the generated code's own imports. Restore the directory name — see `repos/README.md`.
A repo you dropped into `repos/` does not appear	Either it has no `.s3targets.json`, or the console has not restarted since you wrote one. A manifest that exists but cannot be parsed *raises at import* rather than being skipped, so a silent absence means "no manifest", not "bad manifest".
`/admin` 403s	You are logged in as an engineer or a tester. It is manager-only server-side, not hidden by the UI. Log in as **Manager** (9000).